BTX: Efficient Batch Threshold Encryption with Quasi-Linear Decryption

Abstract

Batched threshold encryption (BTE) enables a committee of servers to jointly decrypt any chosen subset of ciphertexts from a large public pool, while all remaining ciphertexts stay private. BTEis a key building block for *encrypted mempools*, where transactions are encrypted until block inclusion to mitigate *maximal extractable value* (MEV). However, existing BTEconstructions require $O(B^2)$ pairing computations to decrypt a batch of B ciphertexts, making them impractical for the few-second block intervals of modern blockchains.

In this paper, we present a new BTEconstruction based on a power-basis structure over bilinear groups with a single omitted public power. Our construction requires only a one-time trusted setup and supports decryption of arbitrary batches without per-epoch or per-block key generation. The key technical contribution is an observation that the cross-term computation during decryption forms a polynomial convolution, which can be evaluated using a Fast Fourier Transform (FFT). This reduces the dominant decryption cost from $O(B^2)$ to quasi-linear $O(B \log B)$ pairing-equivalent operations. Moreover, the entire convolution-based computation can be performed *before* the threshold reconstruction completes, further reducing the latency-critical online phase to only $O(B)$ pairings.

[Todo: Add concrete numbers once evaluation is done.]

1 Introduction

[Todo: Flesh out into full prose following the 5-block arc.]

Block 1: The problem.

- *Batched threshold encryption* (BTE) enables a committee of servers to jointly decrypt any chosen subset of ciphertexts from a larger pool, while keeping all remaining ciphertexts private [5, 1, 4, 3].
- Primary application: *encrypted mempools* for MEV mitigation. Users encrypt transactions under a shared public key; validators collectively decrypt only the transactions included in a finalized block.
- BTE must be scalable: decryption communication should grow sublinearly in the batch size and be independent of the pool size.

Block 2: Existing approaches and their limitations.

- Choudhuri et al. [5]: achieves $O(t)$ decryption communication but requires a fresh MPC setup per epoch/block.
- Agarwal et al. [1] and Choudhuri et al. [6]: one-time DKG setup, but ciphertexts are tied to a specific batch identity (block label). Ciphertext can only be decrypted in its designated block — inflexible for real mempools.

- BEAT-MEV [4]: first epochless BTEwith concrete efficiency. However:
  - Decryption requires B^2 bilinear pairings — far too slow for modern blockchains (e.g., approximately 2 minutes for $B = 512$).
  - Clients must select indices from a small domain; collisions require expensive mitigation.
- BEAST-MEV [3]: extends BEAT-MEV with silent setup, but remains $O(B^2)$ in computation.

Block 3: Our results.

- In this paper, we present a new BTEconstruction based on a power-basis structure over bilinear groups. The construction is simple: the public parameters consist of a "punctured" powers-of-tau CRS in $\mathbb{G}_2$ (with one power omitted), and encryption is a standard hybrid ciphertext using the omitted power in the target group.
- **Quasi-linear decryption.** We observe that the cross-term computation during batch decryption forms a polynomial convolution. Using FFT/NTT, this reduces the dominant decryption cost from $O(B^2)$ to $O(B \log B)$.
- **Pre-computation.** The entire convolution can be computed before the threshold reconstruction completes. The latency-critical online phase requires only $O(B)$ pairings and $O(t)$ group operations.
- **No per-ciphertext indices.** Unlike BEAT-MEV, our construction does not require clients to select batch indices at encryption time. The ciphertext's role in the batch is determined purely by its position in the ordered batch at decryption time. This eliminates the collision problem entirely.
- **One-time setup.** The construction requires only a one-time trusted setup (or DKG) and supports ongoing encryption and decryption without per-block ceremonies.

Evaluation. [Todo: Add evaluation summary once benchmarks are available.]

Paper organization. The rest of the paper is organized as follows. We present a technical overview of our construction in §2. We describe preliminaries in §3 and formalize the BTE definition in §4. We present our construction in detail in §5 and analyze its correctness in §7. We describe our computational optimizations in §6. We present our evaluation in §8. We discuss practical considerations in §9 and related work in §10.

2 Technical Overview

[Todo: Expand these bullet points into full prose with the strawman-to-solution arc.]

Notation. We use $[\![x]\!]_1, [\![x]\!]_2, [\![x]\!]_T$ to denote group elements in $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ respectively, and $\circ$ for the bilinear pairing, so that $[\![a]\!]_1 \circ [\![b]\!]_2 = [\![ab]\!]_T$. Let B_{max} denote the maximum supported batch size and $B \leq B_{\mathsf{max}}$ the actual batch size.

Starting point: BEAT-MEV.

- BEAT-MEV [4] builds BTEfrom two ingredients: a *threshold homomorphic encryption* (THE) scheme and a *key-homomorphic puncturable PRF* (KH-PPRF). Each client samples a PRF key, punctures it at a chosen index, and encrypts the key under THE. The committee threshold-decrypts the aggregate key and uses it to "patch" each punctured key, recovering all messages.

- The bottleneck is decryption: for each of the B messages, the committee evaluates the PRF at B points, yielding B^2 pairings total.

Key idea: power-basis construction.

- We take a different approach. Instead of using KH-PPRF as a black box, we design a direct construction using a "punctured" powers-of-tau CRS.
- The public parameters include all powers $[\![\tau]\!]_2, [\![\tau^2]\!]_2, \ldots, [\![\tau^{2B_{\mathsf{max}}}]\!]_2$ in $\mathbb{G}_2$ *except* the middle power $[\![\tau^{B_{\mathsf{max}}+1}]\!]_2$. The target group element $[\![\tau^{B_{\mathsf{max}}+1}]\!]_T$ is published separately.
- Encryption of a message m: sample $r \leftarrow\$ \mathbb{Z}_p$ and output $\mathsf{ct} := ([\![r]\!]_1,\ m \oplus \mathsf{H}([\![r \cdot \tau^{B_{\mathsf{max}}+1}]\!]_T))$.
- Intuitively, the omitted power in $\mathbb{G}_2$ prevents a third party from computing the encryption pad from the ciphertext alone, while the threshold servers can reconstruct it via the aggregate key.

Threshold decryption.

- The secret τ^k (for $k \in [B_{\mathsf{max}}]$) is Shamir-shared among the servers. Given a batch $(\mathsf{ct}_1, \ldots, \mathsf{ct}_B)$, each server j computes a single aggregate share: $\mathsf{sbk}_j := \sum_{k=1}^{B} \langle \tau^k \rangle_j \cdot [\![r_k]\!]_1$, and sends it to the combiner. Communication is $O(1)$ per server — independent of B.
- The combiner reconstructs $\mathsf{sbk} = \sum_{k=1}^{B} [\![r_k \tau^k]\!]_1$ using Lagrange interpolation.

Cross-term cancellation.

- For each slot $i \in [B]$, the combiner computes:

$$\alpha_i := \mathsf{sbk} \circ [\![\tau^{B_{\mathsf{max}}+1-i}]\!]_2 = \sum_{k=1}^{B} [\![r_k \tau^{k+B_{\mathsf{max}}+1-i}]\!]_T,$$

$$\beta_i := \sum_{\substack{k=1 \\ k \neq i}}^{B} [\![r_k]\!]_1 \circ [\![\tau^{B_{\mathsf{max}}+1-i+k}]\!]_2.$$

- Subtracting gives $\gamma_i := \alpha_i - \beta_i = [\![r_i \cdot \tau^{B_{\mathsf{max}}+1}]\!]_T$, which is exactly the encryption pad for message m_i.
- The key observation: the exponent $B_{\mathsf{max}} + 1 - i + k$ in β_i never equals $B_{\mathsf{max}} + 1$ when $k \neq i$, so every required $\mathbb{G}_2$ element is present in the CRS.

FFT acceleration of the cross-terms.

- The β_i values depend only on the *relative offset* $k - i$. This means the vector $(\beta_1, \ldots, \beta_B)$ is a contiguous coefficient window of a bilinear polynomial product — a *middle-product* computation.
- Since one polynomial (the CRS powers) is fixed, its FFT representation can be precomputed once. The variable polynomial (from the ciphertext batch) is transformed per batch. The middle product is then computed in $O(B \log B)$ time using FFT/NTT.
- Alternative: a Karatsuba-style recursive decomposition gives subquadratic complexity without requiring a target-group FFT, which may be preferable for moderate batch sizes.

Pre-computation and latency decomposition.

- The β_i values depend only on the ciphertext batch and the public CRS — not on the threshold output sbk. Therefore, the entire FFT-based cross-term computation can proceed in parallel with the threshold reconstruction.
- The latency-critical online phase (after sbk is available) requires only: (1) reconstruct sbk from shares, (2) compute B pairings for α_i, (3) subtract the pre-computed β_i values, and (4) hash and unmask the messages.

3 Preliminaries

Notations. We use λ to denote a computational security parameter, $[a, b]$ to represent the set of integers $\{a, a+1, \ldots, b\}$ and $[n] := [1, n]$, $x \leftarrow\$ S$ to denote that x is sampled uniformly at random from set S. We use bold letters to indicate vectors. For a vector $\mathbf{v}$ of length n, we use v_i to denote the ith element where $i \in [n]$. By $\mathsf{poly}(\lambda)$ and $\mathsf{negl}(\lambda)$, we mean the class $\lambda^{O(1)}$ and $\frac{1}{\lambda^{\omega(1)}}$. We use PPT to denote probabilistic $\mathsf{poly}(\lambda)$-time Turing Machines.

3.1 Bilinear Groups

We follow the notation used in [8]. A bilinear group $\mathcal{BG}$ is a set of three groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ of prime order p, with a (non-degenerate) bilinear map or pairing e. This map takes one element from $\mathbb{G}_1$ and one element from $\mathbb{G}_2$ and produces an element in $\mathbb{G}_T$. The groups $\mathbb{G}_1$ and $\mathbb{G}_2$ are called the source groups, and $\mathbb{G}_T$ is the target group.

The groups $\mathbb{G}_1, \mathbb{G}_2$ have generators g_1, g_2, and we use the notation $[\![x]\!]_s$ to represent $x \cdot g_s$ in the group $\mathbb{G}_s$ for $s \in \{1, 2, T\}$, where $x \in \mathbb{Z}_p$ and the generator of $\mathbb{G}_T$ is $g_T = e(g_1, g_2)$. The group operation is additive, and therefore $[\![x]\!]_s + [\![y]\!]_s = [\![x + y]\!]_s$.

We represent the pairing operation $e([\![x]\!]_1, [\![y]\!]_2)$ as $[\![x]\!]_1 \circ [\![y]\!]_2 = [\![y]\!]_2 \circ [\![x]\!]_1 = [\![x \cdot y]\!]_T$. This notation composes naturally with linear operations. For vectors $\mathbf{u} \in \mathbb{G}_1^n, \mathbf{v} \in \mathbb{G}_2^n$, we have $\mathbf{u}^T \circ \mathbf{v} = [\![u_1 v_1 + \cdots + u_n v_n]\!]_T$.

3.2 Shamir Secret Sharing

We use a (t, N) Shamir secret sharing scheme over $\mathbb{Z}_p$. For any secret $x \in \mathbb{Z}_p$, the sharing procedure $\mathsf{Share}(x, N, t)$ produces shares $(\langle x \rangle_1, \ldots, \langle x \rangle_N)$ such that any subset $T \subseteq [N]$ of size $|T| = t$ determines Lagrange coefficients $(\lambda_j)_{j \in T}$ satisfying $\sum_{j \in T} \lambda_j \langle x \rangle_j = x$.

3.3 Hardness Assumptions

Assumption 3.1 (Decisional n-Power Diffie–Hellman-1 ($\mathsf{nDH\text{-}1}$))**.** *Let* GrpGen *be an asymmetric bilinear group generator. The* $\mathsf{nDH\text{-}1}$ *problem is hard for* GrpGen *if the following two distributions* $\mathcal{D}_0$ *and* $\mathcal{D}_1$ *are computationally indistinguishable:*

$$\mathcal{D}_0 = \Big(\{[\![x^i]\!]_1\}_{i \in [n]}, \{[\![x^i]\!]_2\}_{i \in [2n+1] \setminus \{n+1\}}, [\![y]\!]_1, [\![x^{n+1} \cdot y]\!]_T \Big),$$

$$\mathcal{D}_1 = \Big(\{[\![x^i]\!]_1\}_{i \in [n]}, \{[\![x^i]\!]_2\}_{i \in [2n+1] \setminus \{n+1\}}, [\![y]\!]_1, [\![z]\!]_T \Big),$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [\![1]\!]_1, [\![1]\!]_2, [\![1]\!]_T, p, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$ *and* $(x, y, z) \leftarrow\$ \mathbb{Z}_p^*$.

[Todo: Add additional assumptions as needed for the full security proof.]

4 Batched Threshold Encryption: Definition

We formalize the syntax and security of *Batched Threshold Encryption* (BTE), following the definition in [4].

Definition 4.1 (Batched Threshold Encryption)**.** A batched threshold encryption scheme BTE consists of the following PPT algorithms and protocols:

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^{B_{\mathsf{max}}})$: The randomized setup algorithm takes as input the security parameter λ and the maximum batch size B_{max} and outputs public parameters pp, which is an implicit input to all subsequent interfaces. The pp also contains an explicit description of an index space $[n]$.
- $(\mathsf{mpk}, \{\mathsf{sk}_j\}_{j\in[N]}) \leftarrow \mathsf{KeyGen}(N, t)$: The randomized key-generation algorithm takes as input the total number of servers N and the corruption threshold t, and outputs a master public key mpk to all, and the j-th secret key sk_j to server j.
- $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{m}, i^*)$: The randomized encryption algorithm takes as input a master public key mpk, a message m, and a batch index $i^* \in [n]$, and outputs a ciphertext ct. We require that ct contains an explicit description of the index i^*.
- $\{\widetilde{\mathsf{m}}_\ell\}_{\ell\in[B]} \leftarrow \Pi\text{-}\mathsf{BDec}(\mathsf{mpk}, \{\mathsf{ct}_\ell\}_{\ell\in[B]}, \langle \mathsf{sk}_j \rangle)$: This is an interactive protocol among all $[N]$ servers for batch decryption, where the jth server has a private input sk_j, and all servers have common input mpk and a batch of ciphertexts $\{\mathsf{ct}_\ell\}_{\ell\in[B]}$ where $B \le B_{\mathsf{max}}$. The protocol outputs the decrypted messages $\{\widetilde{\mathsf{m}}_\ell\}_{\ell\in[B]}$ where each $\widetilde{\mathsf{m}}_\ell$ is either a valid message or $\perp$.

We require the BTE scheme to satisfy the *correctness*, $\mathsf{IND\text{-}CCA}$ *security*, and *robustness* properties. We formalize these properties in Theorems 4.2 to 4.4 and describe them next.

Remark (Position-indexed encryption). In the general BTEdefinition, the encryption algorithm takes an explicit batch index i^*. Our construction (§5) instantiates a simpler variant where encryption does not require an index: the ciphertext's role in the batch is determined entirely by its position in the ordered batch at decryption time. For the purpose of Theorems 4.2 to 4.4, we set $i^* = i$ (the batch position) and note that i^* is not embedded in the ciphertext. This eliminates the collision problem present in constructions like BEAT-MEV [4].

Correctness. The *correctness* property guarantees that correctly encrypted ciphertexts successfully decrypt to the original encrypted messages.

Definition 4.2 (Correctness of BTE)**.** A batched threshold encryption scheme BTE is correct if for all $N \in \mathbb{N}, t \le N$, $\lambda \in \mathbb{N}, B_{\mathsf{max}} \in \mathbb{N}, B \le B_{\mathsf{max}}$, all messages $(\mathsf{m}_1, \mathsf{m}_2, \ldots, \mathsf{m}_B) \in \mathcal{M}^B$, the following probability is 1:

$$\Pr\left[\begin{array}{l} \forall\, i \in [B] \\ \widetilde{\mathsf{m}}_i = \mathsf{m}_i \end{array} : \begin{array}{r} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^{B_{\mathsf{max}}}) \\ \left(\begin{array}{l}\mathsf{mpk},\\ \{\mathsf{sk}_j\}_{j\in[N]}\end{array}\right) \leftarrow \mathsf{KeyGen}(N, t) \\ \{\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{m}_i, i)\}_{i\in[B]} \\ \{\widetilde{\mathsf{m}}_i\}_{i\in[B]} \leftarrow \Pi\text{-}\mathsf{BDec}\left(\begin{array}{l}\mathsf{mpk}, \{\mathsf{ct}_i\}_{i\in[B]},\\ \langle \mathsf{sk}_j \rangle_{j\in[N]}\end{array}\right) \end{array}\right]$$

$\mathsf{IND\text{-}CCA}$ **Security.** The $\mathsf{IND\text{-}CCA}$ security for BTE guarantees that an adversary cannot distinguish between encryptions of two equal-length messages, even when it can invoke the batch decryption protocol ($\Pi\text{-}\mathsf{BDec}$) on arbitrary batches of its choice not containing the challenge ciphertext. Security holds even when the adversary statically corrupts servers up to t.

Game $\mathsf{Expt}^{\mathsf{BTE\text{-}CCA}}_{\mathcal{A},b}(N,t,\lambda,B_{\mathsf{max}})$:

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^{B_{\mathsf{max}}})$
$\mathcal{Q}_{\mathsf{BDec}} \leftarrow \emptyset$
$\mathcal{C} \leftarrow \mathcal{A}(\mathsf{pp})$
$(\mathsf{mpk}, \{\mathsf{sk}_j\}_{j\in[N]}) \leftarrow \mathsf{KeyGen}(N,t)$
$(\mathsf{m}_0, \mathsf{m}_1, i^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{BDec}}}(\mathsf{mpk}, \{\mathsf{sk}_j\}_{j\in\mathcal{C}})$
$\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{m}_b, i^*)$
$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{BDec}}}(\mathsf{ct}^*)$
if $|\mathcal{C}| \le t \ \wedge \ \mathsf{ct}^* \notin \mathcal{Q}_{\mathsf{BDec}}$:
    return b'
return 0

Oracle $\mathcal{O}_{\mathsf{BDec}}(\{\mathsf{ct}_i\}_{i\in[B]})$:

// Π-BDec is run among all $[N]$ parties where $\mathcal{A}$ controls the parties in $\mathcal{C}$.
Let $\{\mathsf{m}_i\}_{i\in[B]} \leftarrow \Pi\text{-}\mathsf{BDec}(\mathsf{mpk}, \{\mathsf{ct}_i\}_{i\in[B]}, \langle \mathsf{sk}_j \rangle)$
$\mathcal{Q}_{\mathsf{BDec}} \leftarrow \mathcal{Q}_{\mathsf{BDec}} \cup \{\mathsf{ct}_i\}_{i\in[B]}$
return $\{\mathsf{m}_i\}_{i\in[B]}$

Game $\mathsf{Expt}^{\mathsf{BTE\text{-}rob}}_{\mathcal{A}}(N,t,\lambda,B_{\mathsf{max}})$:

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^{B_{\mathsf{max}}})$
$\mathcal{C} \leftarrow \mathcal{A}(\mathsf{pp})$
if $|\mathcal{C}| > t$: **return** 0
$(\mathsf{mpk}, \{\mathsf{sk}_j\}_{j\in[N]}) \leftarrow \mathsf{KeyGen}(N,t)$
$(B, S, \{\mathsf{m}_i\}_{i\in[B]\setminus S}) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{BDec}}}(\mathsf{mpk}, \{\mathsf{sk}_j\}_{j\in[N]})$
if $B > B_{\mathsf{max}} \vee S \not\subset [B]$: **return** 0
for all $i \in [B] \setminus S$:
    $\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{m}_i, i)$
$\{\mathsf{ct}_i\}_{i\in S} \leftarrow \mathcal{A}(\{\mathsf{ct}_i\}_{i\in[B]\setminus S})$
$\{\widetilde{\mathsf{m}_i}\}_{i\in[B]} \leftarrow \mathcal{O}_{\mathsf{BDec}}(\{\mathsf{ct}_i\}_{i\in[B]})$
if $\exists\ i \in [B] \setminus S$ s.t. $\widetilde{\mathsf{m}_i} \neq \mathsf{m}_i$:
    return 1
return 0

Figure 1: Security game $\mathsf{Expt}^{\mathsf{BTE\text{-}CCA}}_{\mathcal{A},b}$ and robustness game $\mathsf{Expt}^{\mathsf{BTE\text{-}rob}}_{\mathcal{A}}$ for a BTE scheme.

Definition 4.3 (IND-CCA Security of BTE)**.** Let $\mathsf{Expt}^{\mathsf{BTE\text{-}CCA}}_{\mathcal{A},b}$ be the game as defined in Fig. 1. A batched threshold encryption scheme BTE is IND-CCA secure if for all $N \in \mathbb{N}, t < N, B_{\mathsf{max}} \in \mathbb{N}$, and all stateful PPT adversaries $\mathcal{A}$, we have

$$\left| \Pr\left[\mathsf{Expt}^{\mathsf{BTE\text{-}CCA}}_{\mathcal{A},0}(N,t,\lambda,B_{\mathsf{max}}) \Rightarrow 1\right] - \Pr\left[\mathsf{Expt}^{\mathsf{BTE\text{-}CCA}}_{\mathcal{A},1}(N,t,\lambda,B_{\mathsf{max}}) \Rightarrow 1\right] \right| = \mathsf{negl}(\lambda)$$

Robustness. The *robustness* property prevents an adversary $\mathcal{A}$ that can corrupt parties up to t from invoking the batch decryption protocol (Π-BDec) on malformed ciphertexts to disrupt decryption of correctly generated ciphertexts.

Definition 4.4 (Robustness of BTE)**.** Let $\mathsf{Expt}^{\mathsf{BTE\text{-}rob}}_{\mathcal{A}}$ be the game as defined in Fig. 1. A batched threshold encryption scheme BTE is robust if for all $N \in \mathbb{N}, t < \lfloor N/2 \rfloor, B_{\mathsf{max}} \in \mathbb{N}$, we have:

$$\Pr[\mathsf{Expt}^{\mathsf{BTE\text{-}rob}}_{\mathcal{A}}(N,t,\lambda,B_{\mathsf{max}}) \Rightarrow 1] = \mathsf{negl}(\lambda)\ .$$

Remark (Efficiency of BTE). We are interested in BTE schemes where the total communication complexity of Π-BDec, across all N servers, is $o(B)$ — meaning that it scales sublinearly in the batch size B.

5 Construction

We now describe our BTEconstruction in detail. We summarize the full protocol in Fig. 2 and describe each algorithm next.

5.1 Setup

The setup algorithm generates a bilinear group and samples a secret scalar $\tau \in \mathbb{Z}_p$. It then publishes a structured common reference string (CRS) consisting of powers of τ in $\mathbb{G}_2$, with one critical power

omitted:

$$[\![\tau]\!]_2, [\![\tau^2]\!]_2, \ldots, [\![\tau^{B_{\max}}]\!]_2, \quad \cancel{[\![\tau^{B_{\max}+1}]\!]_2}, \quad [\![\tau^{B_{\max}+2}]\!]_2, \ldots, [\![\tau^{2B_{\max}}]\!]_2.$$

That is, all powers $\tau, \tau^2, \ldots, \tau^{2B_{\max}}$ in $\mathbb{G}_2$ are published *except* the middle power $\tau^{B_{\max}+1}$. Moreover, the target group element $[\![\tau^{B_{\max}+1}]\!]_T$ is published, which serves as the encryption key.

Intuitively, publishing all $\mathbb{G}_2$ powers except $[\![\tau^{B_{\max}+1}]\!]_2$ allows anyone to compute pairings involving arbitrary differences of powers (needed for the cross-term cancellation in decryption), but prevents direct computation of $[\![r{\cdot}\tau^{B_{\max}+1}]\!]_T$ from a ciphertext $[\![r]\!]_1$ alone. The element $[\![\tau^{B_{\max}+1}]\!]_T$ in the target group serves as a "one-way" encryption key: given $r \in \mathbb{Z}_p$, one can compute $[\![r{\cdot}\tau^{B_{\max}+1}]\!]_T$, but given only $[\![r]\!]_1$, one cannot.

5.2 Key Generation

For each exponent $k \in [B_{\max}]$, the setup Shamir-shares the scalar τ^k among the N servers with threshold t:

$$\big(\langle\tau^k\rangle_1, \ldots, \langle\tau^k\rangle_N\big) \leftarrow \mathsf{Share}(\tau^k, N, t).$$

Server j receives $\mathsf{sk}_j := \big(\langle\tau\rangle_j, \langle\tau^2\rangle_j, \ldots, \langle\tau^{B_{\max}}\rangle_j\big)$.

Note that the master public key mpk consists of the public parameters pp themselves. Unlike BEAT-MEV [4], where each client generates a per-ciphertext PRF key, our construction uses the global CRS structure directly.

5.3 Encryption

To encrypt a message m, the client samples fresh randomness $r \leftarrow\!\$\ \mathbb{Z}_p$ and outputs

$$\mathsf{ct} := \Big(\underbrace{[\![r]\!]_1}_{\mathsf{ct.first}},\ \underbrace{m \oplus \mathsf{H}\big([\![r \cdot \tau^{B_{\max}+1}]\!]_T\big)}_{\mathsf{ct.second}}\Big),$$

where $\mathsf{H} : \mathbb{G}_T \to \{0,1\}^\lambda$ is a hash function modeled as a random oracle.

The first component is a random $\mathbb{G}_1$ element, and the second component is a one-time-pad encryption of m using the hash of $[\![r \cdot \tau^{B_{\max}+1}]\!]_T$ as the pad. The client computes $[\![r \cdot \tau^{B_{\max}+1}]\!]_T$ by exponentiating the published target group element $[\![\tau^{B_{\max}+1}]\!]_T$ by r.

We emphasize that encryption does *not* require the client to choose a batch index. The ciphertext's role in the batch is determined entirely by its position at decryption time (see Section 4).

5.4 Batch Decryption

The batch decryption protocol Π-BDec takes as input a batch of $B \le B_{\max}$ ciphertexts $(\mathsf{ct}_1, \ldots, \mathsf{ct}_B)$ and proceeds in two rounds.

Round 1: Pre-decryption shares (Π-BDec_1). Each server j uses its secret key shares $\mathsf{sk}_j = (\langle\tau\rangle_j, \ldots, \langle\tau^{B_{\max}}\rangle_j)$ to compute a single aggregate pre-decryption share:

$$\mathsf{sbk}_j := \sum_{k=1}^{B} \langle\tau^k\rangle_j \cdot \mathsf{ct}_k.\mathsf{first} \ \in \mathbb{G}_1.$$

This is a multi-scalar multiplication (MSM) of the B ciphertext elements $\mathsf{ct}_k.\mathsf{first} = [\![r_k]\!]_1$ with the server's scalar shares $\langle\tau^k\rangle_j$. Server j sends only sbk_j to the combiner. The communication per server is a single $\mathbb{G}_1$ element, independent of B.

Round 2: Combine and decrypt (Π-BDec_2). Given shares from a threshold set $T \subseteq [N]$ with $|T| = t$, the combiner first reconstructs the aggregate key:

$$\mathsf{sbk} := \sum_{j \in T} \lambda_j \cdot \mathsf{sbk}_j.$$

By the linearity of Lagrange interpolation and the structure of the shares, this yields

$$\mathsf{sbk} = \sum_{k=1}^{B} [\![r_k \cdot \tau^k]\!]_1.$$

The combiner then decrypts each message m_i for $i \in [B]$ as follows:

1. Compute $\alpha_i := \mathsf{sbk} \circ [\![\tau^{B_{\mathsf{max}}+1-i}]\!]_2$.
2. Compute $\beta_i := \sum_{\substack{k=1 \\ k \neq i}}^{B} \mathsf{ct}_k.\mathsf{first} \circ [\![\tau^{B_{\mathsf{max}}+1-i+k}]\!]_2$.
3. Set $\gamma_i := \alpha_i - \beta_i$.
4. Output $m_i := \mathsf{ct}_i.\mathsf{second} \oplus \mathsf{H}(\gamma_i)$.

The α_i terms require one pairing each (or a single multi-pairing of size B if batched). The β_i terms involve the cross-term computation, which is the subject of the optimization in §6.

Availability of public powers. The exponent in β_i is $B_{\mathsf{max}} + 1 - i + k$. For $k \neq i$ and $1 \leq i, k \leq B \leq B_{\mathsf{max}}$, this exponent ranges over $\{1, 2, \ldots, 2B_{\mathsf{max}}\} \setminus \{B_{\mathsf{max}} + 1\}$, so every required $\mathbb{G}_2$ element is present in the CRS.

6 Computational Optimizations

The main computational bottleneck in the decryption procedure is the evaluation of the cross-term vector $(\beta_1, \ldots, \beta_B)$, where for each $i \in [B]$,

$$\beta_i := \sum_{\substack{k=1 \\ k \neq i}}^{B} \mathsf{ct}_k.\mathsf{first} \circ [\![\tau^{B_{\mathsf{max}}+1-i+k}]\!]_2.$$

A naive implementation computes each β_i using $B - 1$ pairings, for a total of $B(B - 1)$ pairings across all slots. In contrast, the α_i terms require only B pairings (one per slot) and depend on the threshold output sbk. In this section, we present two approaches to reduce the cross-term computation from quadratic to subquadratic.

6.1 Reduction to a Middle Product

We first observe that the β_i values form a contiguous coefficient window of a bilinear polynomial product. Define the $\mathbb{G}_1$-valued polynomial

$$A(X) := \sum_{k=0}^{B-1} A_k X^k, \qquad A_k := \mathsf{ct}_{k+1}.\mathsf{first} \in \mathbb{G}_1,$$

$\underline{\mathsf{Setup}(1^\lambda, 1^{B_{\mathsf{max}}})\textbf{:}}$

$n \coloneqq B_{\mathsf{max}}$
$(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [\![1]\!]_1, [\![1]\!]_2, [\![1]\!]_T, p, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$
$\tau \leftarrow\!\!\$\ \mathbb{Z}_p$
return $\mathsf{pp} \coloneqq \big(n,\ [\![\tau^{n+1}]\!]_T,\ \{[\![\tau^i]\!]_2\}_{i\in[2n]\setminus\{n+1\}}\big)$

$\underline{\mathsf{KeyGen}(N, t, \mathsf{pp})\textbf{:}}$

// Shamir-share each power τ^k among N servers
for $k \in [n]$:
  $\{\langle\tau^k\rangle_j\}_{j\in[N]} \leftarrow \mathsf{Share}(\tau^k, N, t)$
for $j \in [N]$:
  $\mathsf{sk}_j \coloneqq \{\langle\tau^k\rangle_j\}_{k\in[n]}$
$\mathsf{mpk} \coloneqq \mathsf{pp}$
return $(\mathsf{mpk}, \{\mathsf{sk}_j\}_{j\in[N]})$

$\underline{\mathsf{Enc}(\mathsf{mpk}, m)\textbf{:}}$

$r \leftarrow\!\!\$\ \mathbb{Z}_p$
$\mathsf{pad} \coloneqq r \cdot [\![\tau^{n+1}]\!]_T$
$\mathsf{ct.first} \coloneqq [\![r]\!]_1$
$\mathsf{ct.second} \coloneqq m \oplus \mathsf{H}(\mathsf{pad})$
return $\mathsf{ct} \coloneqq (\mathsf{ct.first},\ \mathsf{ct.second})$

$\underline{\Pi\text{-}\mathsf{BDec}_1(\mathsf{mpk}, \{\mathsf{ct}_\ell\}_{\ell\in[B]}, \langle\mathsf{sk}_j\rangle)\textbf{:}}$

// Server j: aggregate pre-decryption share
$\mathsf{sbk}_j \coloneqq \sum_{k=1}^{B} \langle\tau^k\rangle_j \cdot \mathsf{ct}_k.\mathsf{first}$
return sbk_j *// one $\mathbb{G}_1$ element*

$\underline{\Pi\text{-}\mathsf{BDec}_2(\{\mathsf{sbk}_j\}_{j\in[N]}, \{\mathsf{ct}_\ell\}_{\ell\in[B]})\textbf{:}}$

// Threshold reconstruction
Select $T \subseteq [N]$ with $|T| = t$
$\{L_j\}_{j\in T} \leftarrow \mathsf{Lagrange}(T, t)$
$\mathsf{sbk} \coloneqq \sum_{j\in T} L_j \cdot \mathsf{sbk}_j$

// Compute α_i (threshold-dependent)
for $i \in [B]$:
  $\alpha_i \coloneqq \mathsf{sbk} \circ [\![\tau^{n+1-i}]\!]_2$

// Compute β_i (batch-dependent; can be pre-computed)
for $i \in [B]$:
  $\beta_i \coloneqq \sum_{k\in[B],\, k\neq i} \mathsf{ct}_k.\mathsf{first} \circ [\![\tau^{n+1-i+k}]\!]_2$

// Decrypt
for $i \in [B]$:
  $\gamma_i \coloneqq \alpha_i - \beta_i$
  $\widetilde{m}_i \coloneqq \mathsf{ct}_i.\mathsf{second} \oplus \mathsf{H}(\gamma_i)$
return $\{\widetilde{m}_i\}_{i\in[B]}$

Figure 2: Our power-basis BTEconstruction. The public parameters consist of powers of τ in $\mathbb{G}_2$ with the middle power $[\![\tau^{n+1}]\!]_2$ omitted. The β_i cross-terms depend only on the batch and can be accelerated from $O(B^2)$ to $O(B \log B)$ via FFT (§6).

and define the fixed $\mathbb{G}_2$-valued polynomial $Q(X)$ from the CRS powers in reverse order:

$$Q(X) := \sum_{u=0}^{2B_{\mathsf{max}}-1} q_u X^u,$$

where

$$q_u := \begin{cases} [\![\tau^{2B_{\mathsf{max}}-u}]\!]_2 & \text{if } 2B_{\mathsf{max}} - u \neq B_{\mathsf{max}} + 1, \\ 0 & \text{if } 2B_{\mathsf{max}} - u = B_{\mathsf{max}} + 1. \end{cases}$$

The zero coefficient at position $u = B_{\mathsf{max}} - 1$ corresponds to the omitted power $[\![\tau^{B_{\mathsf{max}}+1}]\!]_2$.

Let $C(X) := A(X) \circ Q(X)$, where coefficient multiplication is given by the bilinear map:

$$\Big(\sum_i a_i X^i\Big) \circ \Big(\sum_j b_j X^j\Big) := \sum_m \Big(\sum_{i+j=m} a_i \circ b_j\Big) X^m.$$

Claim 1. *For each $i \in [B]$, the coefficient of $X^{B_{\mathsf{max}}+i-2}$ in $C(X)$ equals β_i.*

Proof. The coefficient of $X^{B_{\mathsf{max}}+i-2}$ is $\sum_{k=1}^{B} \mathsf{ct}_k.\mathsf{first} \circ q_{B_{\mathsf{max}}+i-k-1}$. By definition of q_u, we have $q_{B_{\mathsf{max}}+i-k-1} = [\![\tau^{B_{\mathsf{max}}+1-i+k}]\!]_2$ except when $k = i$, in which case the exponent is $B_{\mathsf{max}} + 1$ and the corresponding coefficient is 0. Therefore the sum equals β_i. □

This means the vector $(\beta_1, \ldots, \beta_B)$ is a contiguous block of B coefficients of $C(X)$, starting at degree $B_{\mathsf{max}} - 1$. The problem of evaluating all β_i thus reduces to a *middle-product* computation, where the second input polynomial $Q(X)$ is fixed across batches.

6.2 Centered CRS Representation

The middle-product structure becomes cleaner in a centered coordinate system. Define $h := [\![\tau^{B_{\mathsf{max}}+1}]\!]_2$ (the omitted element) and, for each nonzero offset $d \in \{-B_{\mathsf{max}}, \ldots, -1, 1, \ldots, B_{\mathsf{max}}\}$, define

$$h_d := [\![\tau^{B_{\mathsf{max}}+1+d}]\!]_2.$$

In this notation, the decryption equations become

$$\alpha_i = \mathsf{sbk} \circ h_{-i}, \qquad \beta_i = \sum_{\substack{k=1 \\ k \neq i}}^{B} \mathsf{ct}_k.\mathsf{first} \circ h_{k-i}.$$

The cross-term depends only on the *relative offset* $k - i$, and the effective CRS window for a batch of size B is:

$$h_{-B}, h_{-(B-1)}, \ldots, h_{-1}, \quad 0, \quad h_1, \ldots, h_{B-1}.$$

This centered kernel of size $O(B)$ is the natural input to both FFT-based and Karatsuba-based middle-product algorithms.

6.3 FFT-Based Middle Product

The first approach evaluates the middle product using an FFT-style transform. The idea is to embed the coefficient window into a cyclic convolution of appropriate length:

1. Convert the coefficient vectors of $A(X)$ and the centered kernel to evaluation form with FFT.
2. Perform pointwise pairings in the evaluation domain.
3. Apply IFFT to recover the product coefficients.
4. Read off β_i from the relevant coefficient window.

Since the CRS polynomial $Q(X)$ is fixed, its FFT representation can be precomputed once and reused across batches. This yields an $O(B \log B)$ method for the cross-term computation.

The main attraction of this approach is its asymptotic efficiency. Note that the inverse transform operates in the target group $\mathbb{G}_T$, where group operations are approximately 3–5$\times$ more expensive than in the source groups; this affects the concrete constants but not the asymptotic improvement.

6.4 Karatsuba-Style Middle Product

The second approach computes the middle product recursively using a Karatsuba-style decomposition. The coefficient window is split into overlapping subproblems, and the standard Karatsuba reuse pattern reduces the number of bilinear multiplications below the naive quadratic count.

Since the CRS polynomial is fixed, all required combinations of its coefficients can be precomputed once and stored. The recursive recombination uses only $\mathbb{G}_T$ additions and inversions, while the expensive pairings occur only at the base level. In particular, the final exponentiation step of

the pairing can be deferred through the recursion, yielding better constants for moderate batch sizes.

This gives a subquadratic algorithm — approximately $O(B^{1.585})$ — without requiring a target-group FFT.

6.5 Pre-computation

A key feature of the power-basis construction is that the entire cross-term computation can begin as soon as the ciphertext batch is fixed, before the threshold reconstruction completes.

Offline phase (after batch selection, before threshold completion).

- Each server computes its local share sbk_j.
- The combiner computes all β_i values using the FFT or Karatsuba method.
- All FFT preprocessing for the convolution is performed.

Online phase (after threshold output is available).

- Reconstruct sbk from the threshold shares.
- Compute α_i for all $i \in [B]$ (B pairings).
- Subtract the pre-computed β_i values.
- Hash and unmask the messages.

The FFT-based computation lies entirely in the offline phase. The online phase requires only B pairings for the α_i values, plus $O(t)$ group operations for the threshold reconstruction. This decomposition is important for applications like encrypted mempools, where the latency between batch finalization and message availability must be minimized.

Optional communication tradeoff. If one is willing to increase communication, even the α_i pairings can be moved offline. Each server j can pre-compute $a_{j,i} := \mathsf{sbk}_j \circ h_{-i}$ for all $i \in [B]$, and the combiner derives $\alpha_i = \sum_{j \in T} \lambda_j \cdot a_{j,i}$. This moves all pairing work before the final threshold step, but increases the per-server communication from one $\mathbb{G}_1$ element to a B-length vector of $\mathbb{G}_T$ elements.

6.6 Summary

The decryption procedure naturally separates into a threshold-dependent component (α_i) and a batch-dependent cross-term component (β_i). The latter is a middle-product problem with fixed second input. Two instantiations are available:

- An FFT-based method with $O(B \log B)$ complexity.
- A Karatsuba-style recursive method with approximately $O(B^{1.585})$ complexity but potentially better concrete performance at moderate sizes.

Both methods preserve the surrounding threshold structure and reduce the dominant decryption cost from quadratic to subquadratic.

7 Analysis

7.1 Correctness

Theorem 7.1 (Correctness)**.** *For every ordered batch* $(\mathsf{ct}_1, \ldots, \mathsf{ct}_B)$ *of honestly generated ciphertexts with* $B \le B_{\mathsf{max}}$*, the decryption algorithm recovers all messages correctly.*

Proof. Fix a slot $i \in [B]$. By definition of sbk,

$$\mathsf{sbk} = \sum_{k=1}^{B} [\![r_k \tau^k]\!]_1.$$

Hence

$$\alpha_i = \mathsf{sbk} \circ [\![\tau^{B_{\mathsf{max}}+1-i}]\!]_2 = \sum_{k=1}^{B} [\![r_k \tau^{k+B_{\mathsf{max}}+1-i}]\!]_T.$$

On the other hand,

$$\beta_i = \sum_{\substack{k=1 \\ k \neq i}}^{B} [\![r_k]\!]_1 \circ [\![\tau^{B_{\mathsf{max}}+1-i+k}]\!]_2 = \sum_{\substack{k=1 \\ k \neq i}}^{B} [\![r_k \tau^{k+B_{\mathsf{max}}+1-i}]\!]_T.$$

Subtracting gives

$$\gamma_i = \alpha_i - \beta_i = [\![r_i \tau^{B_{\mathsf{max}}+1}]\!]_T.$$

Therefore

$$\mathsf{H}(\gamma_i) = \mathsf{H}\big([\![r_i \tau^{B_{\mathsf{max}}+1}]\!]_T\big),$$

which is exactly the mask used in the second component of ct_i. Thus

$$\mathsf{ct}_i.\mathsf{second} \oplus \mathsf{H}(\gamma_i) = m_i.$$

Since i was arbitrary, all messages are recovered. □

7.2 Security

[Todo: Write the full security proof reducing to the nDH-1 assumption. The high-level structure is:]

- The CRS has the same structure as the nDH-1 problem instance: powers of τ in $\mathbb{G}_1$ (used in key shares) and $\mathbb{G}_2$ (published), with the critical power $\tau^{B_{\mathsf{max}}+1}$ missing in $\mathbb{G}_2$.
- The challenge ciphertext $\mathsf{ct}^* = ([\![r^*]\!]_1, m_b \oplus \mathsf{H}([\![r^* \cdot \tau^{B_{\mathsf{max}}+1}]\!]_T))$ maps naturally to the nDH-1 challenge: given $[\![y]\!]_1$ (playing the role of $[\![r^*]\!]_1$), the adversary must distinguish $[\![x^{n+1} \cdot y]\!]_T$ from random.
- Threshold simulation: the adversary corrupts at most $t-1$ servers. The simulator can produce consistent secret key shares for the corrupted servers without knowing τ, using the $\mathbb{G}_1$ powers in the nDH-1 instance.
- The random oracle H is used to extract the adversary's queries, completing the reduction.

Theorem 7.2 (CCA Security)**.** *Assuming the* nDH-1 *assumption holds for the underlying bilinear group generator, our BTEconstruction is* IND-CCA *secure as per Theorem 4.3 in the random oracle model.*

[Todo: Write formal proof.]

7.3 Complexity

For a batch of size $B \leq B_{\mathsf{max}}$ with N servers and threshold t:

- **Encryption:** One scalar multiplication in $\mathbb{G}_T$ and one hash evaluation. Ciphertext size is one $\mathbb{G}_1$ element plus λ bits.
- **Communication per server:** One $\mathbb{G}_1$ element (the share sbk_j), independent of B.
- **Server computation:** $O(B)$ scalar multiplications in $\mathbb{G}_1$ (one MSM of size B).
- **Combiner — online:** $O(t)$ group operations to reconstruct sbk, plus B pairings for (α_i).
- **Combiner — offline:** $O(B \log B)$ operations for the (β_i) cross-terms via FFT (see §6).

8 Implementation and Evaluation

[Todo: Add implementation details and benchmarks. Outline:]

- Implementation language, curve choice (BLS12-381), library used.
- Experimental setup: machine specs, network configuration.
- Metrics: encryption time, pre-decryption share computation, threshold reconstruction, cross-term computation (naive vs. FFT vs. Karatsuba), total decryption latency.
- Comparison with BEAT-MEV [4] and BEAST-MEV [3].
- Batch sizes: $B \in \{64, 128, 256, 512, 1024\}$.
- Online vs. offline latency breakdown.

9 Discussion

Ordered-batch semantics. Our construction is specified for an ordered batch $(\mathsf{ct}_1, \ldots, \mathsf{ct}_B)$. The slot number i used in decryption is the position of the ciphertext inside the selected batch, not an external ciphertext label chosen at encryption time. This design choice eliminates the need for clients to select batch indices, thereby avoiding the collision problem that affects BEAT-MEV [4].

Trusted setup. The construction requires a trusted setup to generate the powers-of-tau CRS and Shamir-share the secret powers. In practice, this can be instantiated using a distributed key generation (DKG) protocol or a universal powers-of-tau ceremony (e.g., [**?**]). We note that the CRS structure is similar to those used in KZG polynomial commitments [**?**] and SNARKs, for which large-scale trusted setup ceremonies already exist.

Relation to BEAT-MEV. Our construction can be viewed as a "de-abstracted" version of the BEAT-MEV framework. BEAT-MEV uses a threshold homomorphic encryption scheme and a key-homomorphic puncturable PRF as black boxes. Our construction effectively opens these black boxes and integrates the PRF evaluation directly into the CRS structure, using the punctured powers-of-tau as a global "punctured PRF key." This integration enables the convolution structure that makes FFT acceleration possible.

Security assumption. The security of our construction relies on the nDH-1 assumption (Assumption 3.1), which informally states that given powers of a secret τ in both source groups (with $\tau^{B_{\max}+1}$ omitted in $\mathbb{G}_2$), and a random $\mathbb{G}_1$ element $[\![y]\!]_1$, it is hard to distinguish $[\![\tau^{B_{\max}+1} \cdot y]\!]_T$ from a random target group element. This assumption is closely related to assumptions used in existing pairing-based BTE constructions [7, 4].

Robustness. [Todo: Discuss how robustness can be achieved by adding NIZK proofs (Schnorr proofs for ciphertext well-formedness and DLEq proofs for server share correctness), following the approach in [4].]

10 Related Work

Threshold encryption for mempool privacy. One prominent approach to achieving mempool privacy is encrypting transactions using threshold encryption schemes [2**?**]. However, a major limitation of traditional threshold encryption is the communication complexity associated with decrypting ciphertexts: many existing solutions incur a communication cost of $O(t \cdot B)$, where $t+1$ is the decryption committee size and B is the batch size.

Identity-based approaches. Shutter [**?**] and Fairblock [**?**] use threshold identity-based encryption to encrypt transactions for an epoch, reducing communication complexity but failing to guarantee *pending transaction privacy*. Similarly, Dötting et al. [**?**] present a constant communication scheme for encryption to the future based on signature-based witness encryption. However, because their witness enables decryption of all ciphertexts within an epoch, pending transaction privacy remains unprotected.

Batched threshold encryption. Several works [5, 6, 1, 4, 3] have formalized the notion of *batched threshold encryption* (BTE), which enables a committee to perform lightweight threshold decryption of an arbitrary batch of ciphertexts while preserving the privacy of ciphertexts outside the batch.

The construction in [5] achieves total decryption communication of $O(t)$ but requires the decryption committee to execute an initial MPC protocol during setup and an additional epoch-specific setup for every block. The constructions in [6, 1] require only a one-time DKG setup, but each ciphertext is bound to a specific batch identity (block label) and can be decrypted only within that batch. This limits flexibility for encrypted mempool applications.

BEAT-MEV [4] introduced the first epochless BTEwith concrete efficiency and no ciphertext-to-block binding. However, decrypting a batch of B ciphertexts requires B^2 bilinear pairings, and the scheme does not natively support weighted validators. BEAST-MEV [3] extends BEAT-MEV with silent setup but remains $O(B^2)$ in computation.

Our construction addresses the quadratic computation bottleneck by reducing the cross-term computation to a polynomial convolution, yielding $O(B \log B)$ decryption cost. Moreover, our construction eliminates the need for per-ciphertext index selection, removing the collision problem that affects BEAT-MEV.

[Todo: Cite and discuss any recent concurrent works.]

References

[1] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors,

Advances in Cryptology – CRYPTO 2025, Part III, volume 16002 of *Lecture Notes in Computer Science*, pages 69–100, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.

[2] Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Report 2022/898, 2022.

[3] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. Beast-mev: Batched threshold encryption with silent setup for mev prevention. *Cryptology ePrint Archive*, 2025.

[4] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. {BEAT-MEV}: Epochless approach to batched threshold encryption for {MEV} prevention. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3457–3476, 2025.

[5] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.

[6] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. Cryptology ePrint Archive, Report 2024/1516, 2024.

[7] Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. Time-lock puzzles with efficient batch solving. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 311–341, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.

[8] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 352–386, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.